Lab 4: Mining Frequent Patterns and Association Rules

Objective

To discover frequent itemsets and generate association rules from transactional data using the `Apriori` and `FP-Growth` algorithms implemented in the `mlxtend` library.

Theory

Association rule mining finds interesting relationships (associations, correlations) between variables in large datasets. It is a fundamental data mining technique used in market basket analysis, recommendation systems, and behavior analytics.

Definitions:

- **Itemset:** A collection of one or more items.
- **Support:** Fraction of transactions containing an itemset.
- **Confidence:** Probability that a rule holds, given the antecedent.
- **Lift:** Measure of how much more often items occur together than expected if they were independent.

Algorithms:

- **Apriori:** Iteratively builds larger frequent itemsets using previously found frequent subsets.
- **FP-Growth:** Constructs an FP-tree to mine itemsets without candidate generation, faster for dense datasets.

Major Functions Used

- `mlxtend.frequent_patterns.apriori()`: Generates frequent itemsets.
- `mlxtend.frequent_patterns.fpmax() / fpgrowth()`: Alternative faster algorithms.
- `mlxtend.frequent_patterns.association_rules()`: Derives rules from frequent itemsets.

Procedure

1. Install and Import Packages

```
conda install -c conda-forge mlxtend -y
# or
pip install mlxtend
```

```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules,
    fpgrowth
```

2. Load and Prepare Transaction Data

We will create a small dataset representing a supermarket transaction list.

```
dataset = [
    ['Milk', 'Bread', 'Butter'],
    ['Bread', 'Eggs', 'Butter', 'Jam'],
    ['Milk', 'Bread', 'Eggs'],
    ['Milk', 'Bread', 'Butter', 'Eggs'],
    ['Bread', 'Butter']
]

# Convert dataset to one-hot encoded DataFrame
from mlxtend.preprocessing import TransactionEncoder

te = TransactionEncoder()
te_ary = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_ary, columns=te.columns_)

print(df.head())
```

Explanation: Each transaction is transformed into a Boolean matrix indicating the presence (True/False) of each item.

3. Apply Apriori Algorithm

```
# Find frequent itemsets with minimum support = 0.4
frequent_itemsets = apriori(df, min_support=0.4, use_colnames=True)
print(frequent_itemsets)
```

Output:

```
   support           itemsets
0      0.8            (Bread)
1      0.6            (Butter)
2      0.6              (Milk)
3      0.6              (Eggs)
4      0.4    (Bread, Butter)
...
```

4. Generate Association Rules

```
rules = association_rules(frequent_itemsets, metric="confidence",
   min_threshold=0.6)
print(rules[['antecedents','consequents','support',
'confidence','lift']])
```

Sample Output:

```
  antecedents     consequents  support  confidence  lift
0    (Bread)          (Butter)    0.4       0.67      1.11
1    (Butter)         (Bread)     0.4       0.67      1.11
2    (Milk, Bread)    (Eggs)      0.4       0.75      1.25
```

Interpretation: The rule *(Bread → Butter)* means that 67% of customers who bought Bread also bought Butter, and the lift > 1 indicates a positive association.

5. Apply FP-Growth Algorithm

```
frequent_fp = fpgrowth(df, min_support=0.4, use_colnames=True)
rules_fp = association_rules(frequent_fp, metric="lift",
   min_threshold=1.0)
print(rules_fp[['antecedents','consequents','support',
'confidence','lift']])
```

Explanation: FP-Growth avoids generating all candidate itemsets like Apriori, making it faster for large transactional datasets.

6. Visualization of Frequent Itemsets

```
import matplotlib.pyplot as plt

frequent_itemsets.sort_values('support', ascending=False,
   inplace=True)
plt.bar(x=range(len(frequent_itemsets)),
   height=frequent_itemsets['support'])
plt.xticks(range(len(frequent_itemsets)),
           [', '.join(x) for x in frequent_itemsets['itemsets']],
   rotation=45)
plt.title("Frequent Itemsets by Support")
plt.ylabel("Support")
plt.show()
```

Observation: Bar plots of support values help quickly identify the most common item combinations.

7. Full Example Script

```python
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules, fpgrowth

dataset = [
    ['Milk', 'Bread', 'Butter'],
    ['Bread', 'Eggs', 'Butter', 'Jam'],
    ['Milk', 'Bread', 'Eggs'],
    ['Milk', 'Bread', 'Butter', 'Eggs'],
    ['Bread', 'Butter']
]

# Convert to one-hot DataFrame
te = TransactionEncoder()
te_ary = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_ary, columns=te.columns_)

# Apriori
freq_ap = apriori(df, min_support=0.4, use_colnames=True)
rules_ap = association_rules(freq_ap, metric="confidence", min_threshold=0.6)
print("Apriori Rules:\n", rules_ap[['antecedents','consequents','support','confidence','lift']])

# FP-Growth
freq_fp = fpgrowth(df, min_support=0.4, use_colnames=True)
rules_fp = association_rules(freq_fp, metric="lift", min_threshold=1.0)
print("\nFP-Growth Rules:\n", rules_fp[['antecedents','consequents','support','confidence','lift']])
```

Listing 3: Complete Apriori and FP-Growth Implementation

Discussion

Both Apriori and FP-Growth discover frequent co-occurrences of items, but FP-Growth is computationally more efficient. These algorithms are crucial in recommendation systems (e.g., "people who bought X also bought Y") and retail analytics.

Exercises

1. Create your own transactional dataset with at least 10 transactions.
2. Apply Apriori and FP-Growth to find itemsets with support ≥ 0.3.
3. Compare the number of rules generated by both algorithms.

4. Visualize top 10 itemsets using support values.
5. Interpret any two meaningful association rules.

Outcome

Students will be able to:

- Apply Apriori and FP-Growth algorithms using `mlxtend`.
- Interpret frequent itemsets, association rules, and evaluation metrics.
- Understand support, confidence, and lift in the context of market basket analysis.